

AUDIOLIBRO Y GUÍA DE ESTUDIO

Cómo se hicieron los diez programas

*Estructuras iterativas y condicionales, explicadas
para escuchar*

14 capítulos · 10 ejercicios

PSeInt y Python

Índice

- 1 Bienvenida: cómo usar este audiolibro
- 2 El método: las tres preguntas
- 3 El diccionario: símbolos, operadores y conectores
- 4 Ejercicio 1: Tabla de multiplicar
- 5 Ejercicio 2: Suma de los primeros N números
- 6 Ejercicio 3: Contador regresivo
- 7 Ejercicio 4: Acumulado hasta cero
- 8 Ejercicio 5: Impresión de patrones
- 9 Ejercicio 6: Clasificador de pares e impares
- 10 Ejercicio 7: Validación de contraseña
- 11 Ejercicio 8: Números primos en un rango
- 12 Ejercicio 9: Promedio con filtrado
- 13 Ejercicio 10: Juego de adivinanza
- 14 Repaso final: los diez errores que debes evitar

Bienvenida: cómo usar este audiolibro

Bienvenido. En los próximos capítulos vas a entender, paso a paso, cómo se construyeron los diez programas de estructuras iterativas y condicionales. No vamos a leer código en voz alta, porque escuchar símbolos es aburrido y no se entiende. Vamos a hacer algo mejor: te voy a explicar cómo se piensa un programa antes de escribirlo.

Este audiolibro está organizado así. Primero, un método de tres preguntas que sirve para cualquier ciclo que escribas en tu vida. Segundo, un diccionario de los símbolos y conectores que aparecen en los programas: qué significa cada uno, para qué sirve y en cuál ejercicio se usó. Y tercero, los diez ejercicios explicados uno por uno.

Un consejo antes de empezar. Si vas escuchando esto mientras vas en el transporte o haciendo otra cosa, no te preocupes por memorizar los símbolos. Concéntrate en las ideas: el acumulador, el centinela, la bandera, los ciclos anidados. Esas cuatro palabras son el noventa por ciento del curso. Los símbolos los puedes consultar después en el documento escrito.

Y una advertencia útil: si en algún momento te pierdes, regresa al capítulo dos. Todo lo demás se apoya en él.

El método: las tres preguntas

Antes de escribir una sola línea de código, un programador se hace tres preguntas. Solamente tres. Si las contestas bien, el programa sale casi solo. Vamos una por una.

Primera pregunta: ¿cuántas veces se repite, y lo sé de antemano?

Esta pregunta decide qué tipo de ciclo vas a usar, y hay tres respuestas posibles.

Respuesta uno: sí, sé el número exacto. Son diez vueltas, o son ene vueltas, o son las que el usuario me dijo al principio. En ese caso usas el ciclo Para, que en Python se llama for. El ciclo Para lleva su propio contador incorporado, así que tú no tienes que preocuparte de aumentarlo.

Respuesta dos: no lo sé, depende de una condición que se va evaluando. En ese caso usas el ciclo Mientras, que en Python se llama while. El Mientras revisa la condición antes de entrar; por eso, si la condición es falsa desde el principio, el cuerpo del ciclo no se ejecuta ni una sola vez.

Respuesta tres: no lo sé, pero estoy seguro de que se tiene que ejecutar al menos una vez. El caso clásico es pedirle un dato al usuario: primero hay que pedirlo, y solo después puedes juzgar si sirve. Para eso existe el ciclo Repetir Hasta Que, que evalúa la condición al final. En Python este ciclo no existe, y más adelante te voy a enseñar cómo se imita.

Segunda pregunta: ¿qué necesito recordar de una vuelta a la siguiente?

Esta es la pregunta que separa a quien entiende de quien solo copia. Piénsalo así: cada vuelta del ciclo es como una hoja en blanco. Si quieres que algo sobreviva de una vuelta a la siguiente, tienes que guardarlo en una variable creada afuera del ciclo.

Hay tres tipos de variables que sirven para recordar, y las vas a ver una y otra vez.

El acumulador: guarda una suma o un total que va creciendo. Empieza en cero.

El contador: guarda cuántas veces pasó algo. También empieza en cero.

La bandera: es una variable de sí o no, que responde a una pregunta como ¿ya acertó el usuario? Empieza valiendo falso, y cambia a verdadero cuando ocurre el evento.

Y aquí va la regla más importante de todo el curso, la que más gente olvida: estas variables se crean y se ponen en cero ANTES de entrar al ciclo, nunca adentro. Si las pones adentro, se borran en cada vuelta y todo tu trabajo se pierde. Repito, porque es la causa número uno de errores: el acumulador se inicializa una sola vez, antes de entrar al ciclo.

Tercera pregunta: ¿cómo termina?

Si no puedes contestar esta pregunta con precisión, acabas de escribir un ciclo infinito, y tu programa se va a quedar colgado para siempre.

La forma de comprobarlo es sencilla: busca dentro del ciclo la línea que modifica la condición de salida. Si esa línea no existe, el ciclo nunca termina. Por ejemplo, si tu condición dice mientras los intentos sean menores que tres, tiene que haber una línea adentro que aumente los intentos. Si se te olvida esa línea, el programa te va a pedir la contraseña eternamente.

Guarda estas tres preguntas. ¿Cuántas veces? ¿Qué recuerdo? ¿Cómo termina? Con eso vas a poder resolver los diez ejercicios y muchos más.

El diccionario: símbolos, operadores y conectores

En este capítulo vamos a ver todos los símbolos que aparecen en los programas. Te voy a decir qué significa cada uno, para qué sirve, y en cuál ejercicio se usó. Este es el capítulo que más conviene tener también por escrito, porque son símbolos y los símbolos se ven mejor de lo que se escuchan.

3.1 La flecha de asignación

El símbolo más común de todos. En PSeInt se dibuja como una flecha que apunta hacia la izquierda, y se escribe con el signo de menor que seguido de un guion. En Python, esa misma flecha se convierte en un simple signo de igual.

Significa: guarda este valor en esta variable. Y aquí está el punto que cuesta entender al principio: aunque en Python se escriba con un signo de igual, NO es el igual de las matemáticas. No dice que dos cosas sean iguales. Es una orden que dice: toma lo que está a la derecha, y guárdalo en la caja que está a la izquierda.

```
PSeInt: suma <- suma + i  
Python: suma = suma + i    (o su forma corta: suma += i)
```

Por eso una línea como `suma flecha suma más i` tiene todo el sentido del mundo, aunque como ecuación matemática sería falsa. Se lee así: toma el valor que ya tenía la suma, súmale `i`, y guarda el resultado otra vez en la suma.

Dónde se usó: en absolutamente todos los ejercicios. Es la instrucción más frecuente de la programación.

3.2 Los comparadores

Los comparadores sirven para hacer preguntas cuya respuesta es verdadero o falso. Siempre aparecen dentro de un Si o dentro de la condición de un ciclo. Son seis.

Símbolo PSeInt	Símbolo Python	Significa	Ejemplo hablado
=	==	es igual a	si el intento es igual a la clave
<>	!=	es distinto de	si el número es distinto de cero
<	<	es menor que	si el intento es menor que el secreto
>	>	es mayor que	si el contador es mayor que cero
<=	<=	es menor o igual que	mientras el divisor por sí mismo sea menor o igual
>=	>=	es mayor o igual que	si la nota es mayor o igual que cinco

Presta atención a la trampa más famosa de la programación. En PSeInt, un solo signo de igual sirve para dos cosas distintas según el contexto. Pero en Python están separados, y esto es importantísimo: un signo de igual solo asigna, guarda un valor. Dos signos de igual seguidos preguntan si algo es igual. Si escribes uno donde va el otro, Python te marca error o, peor, hace algo distinto de lo que querías.

El otro símbolo que conviene grabarse es el de distinto. En PSeInt es el menor que seguido del mayor que. En Python es el signo de admiración seguido del igual. Los dos significan lo mismo: no son iguales.

Dónde se usaron: el distinto de cero, en el ejercicio cuatro, para no sumar el centinela. El mayor o igual que cinco, en el ejercicio nueve, para filtrar las notas reprobatorias. El menor que, en el ejercicio diez, para decidir si la pista es mayor o menor.

3.3 Los conectores lógicos: Y, O, NO

Aquí llegamos a lo que preguntabas: los conectores. Sirven para unir dos o más condiciones en una sola pregunta. Son tres, y elegir mal entre ellos es un error que cuesta caro.

El conector Y, que en Python se escribe `and`, exige que se cumplan TODAS las condiciones. Basta que una sola falle para que el resultado completo sea falso. Piénsalo como un requisito doble: para entrar necesitas boleto Y identificación. Si te falta cualquiera de las dos, no entras.

El conector O, que en Python se escribe or, se conforma con que se cumpla AL MENOS UNA. Solo es falso cuando todas fallan. Piénsalo como una alternativa: puedes pagar con efectivo O con tarjeta. Con cualquiera de las dos, pasas.

El conector NO, que en Python se escribe not, invierte el resultado. Convierte verdadero en falso y falso en verdadero.

```
PSeInt: Mientras intentos < maxIntentos Y acceso = Falso Hacer  
Python: while intentos < MAX_INTENTOS and not acceso:
```

Este ejemplo es del ejercicio siete, el de la contraseña, y vale la pena detenerse porque explica por qué se eligió el conector Y y no el O.

El ciclo debe seguir mientras se cumplan dos cosas a la vez: que todavía queden intentos, Y que el usuario aún no haya acertado. Con el conector Y, basta que una de las dos falle para salir. Si acierta, salimos. Si se acaban los intentos, también salimos. Perfecto.

Ahora imagina que por error usas el conector O. Cuando el usuario acierta, la segunda condición se vuelve falsa, pero la primera sigue siendo verdadera, porque todavía quedan intentos. Y como al conector O le basta con una, el ciclo continuaría. El programa te seguiría pidiendo la contraseña después de haberte dado el acceso. Ese es el tipo de error que solo se detecta razonando, porque el programa no marca ningún fallo: simplemente hace algo absurdo.

El conector O sí se usó, y bien usado, en el ejercicio diez: si el intento es menor que uno O mayor que cien, entonces está fuera del rango. Aquí basta que cualquiera de las dos se cumpla para rechazar el dato. Ese es exactamente el trabajo del O.

Resumen para no equivocarte: cuando pidas que se cumplan varias cosas al mismo tiempo, usa Y. Cuando te baste con cualquiera de varias, usa O.

3.4 El operador MOD, el residuo

MOD es la palabra que se usa en PSeInt; en Python se escribe con el símbolo de porcentaje, aunque no tiene nada que ver con porcentajes. Devuelve el residuo de una división entera, o sea, lo que sobra.

Un ejemplo con números pequeños. Siete entre dos: cabe tres veces y sobra uno. Entonces siete MOD dos es igual a uno. Ocho entre dos: cabe cuatro veces exactas y

no sobra nada. Entonces ocho MOD dos es igual a cero.

Y de ahí sale la regla que usamos en el ejercicio seis: si un número MOD dos da cero, el número es par. Si da uno, es impar. Esa es la forma estándar de detectar números pares en cualquier lenguaje de programación del mundo.

El mismo operador se usó también en el ejercicio ocho, el de los números primos, pero con otro propósito: si un número MOD divisor da cero, significa que la división fue exacta, y por lo tanto ese divisor sí divide al número. Y si un número tiene un divisor distinto de uno y de sí mismo, entonces no es primo.

Fíjate en algo elegante: el mismo operador responde dos preguntas distintas. ¿Es par? y ¿es divisible? En el fondo son la misma pregunta, hecha con divisores diferentes.

3.5 Los operadores aritméticos

Los cuatro de siempre: más para sumar, menos para restar, asterisco para multiplicar, y diagonal para dividir. El único que se presta a confusión es el asterisco, porque en la escuela la multiplicación se escribe con una equis o con un punto, pero en programación siempre es el asterisco.

Y hay una advertencia grande con la división: nunca dividas sin verificar antes que el divisor no sea cero. Dividir entre cero no da un resultado raro, hace que el programa se caiga. Eso lo vas a ver aplicado en el ejercicio nueve.

3.6 Las palabras que arman los ciclos

Estas son las palabras clave de las estructuras. Te las digo con su equivalente en Python.

PSeInt	Python	Para qué sirve
Para ... Hasta ... Con Paso	for ... in range(...)	Repetir un número conocido de veces
Mientras ... Hacer	while ...	Repetir mientras se cumpla una condición
Repetir ... Hasta Que	while True + break	Repetir al menos una vez
Si ... Entonces	if ...	Tomar una decisión
SiNo	else / elif	Qué hacer si la condición no se cumplió
FinSi, FinPara, FinMientras	la sangría	Marcar dónde termina el bloque

Detente en la última fila, porque es la diferencia más grande entre los dos lenguajes. En PSeInt, cada bloque se cierra con una palabra: FinSi, FinPara, FinMientras. En Python no existen esas palabras: lo que marca dónde empieza y dónde termina un bloque es la SANGRÍA, o sea, los espacios que dejas al principio de la línea.

Esto no es decoración. En Python, si mueves una línea cuatro espacios a la izquierda, la sacas del ciclo y el programa hace algo completamente distinto. Es la causa número uno de errores en quienes vienen de otro lenguaje. Un ejemplo real de estos ejercicios: en el ejercicio dos, si la línea que muestra el resultado quedara dentro del ciclo en vez de afuera, el programa imprimiría el total en cada vuelta en lugar de una sola vez al final.

Y una nota sobre la palabra Con Paso. Le dice al ciclo Para de cuánto en cuánto avanza el contador. Si dices Con Paso uno, va de uno en uno. Si dices Con Paso menos uno, va hacia atrás, restando. Eso es lo que se usó en el ejercicio tres para la cuenta regresiva.

3.7 Los tipos de datos

Cuando declaras una variable en PSeInt tienes que decir de qué tipo es. Hay cuatro que usamos.

Entero: números sin decimales. Se usó para contadores, para los números que se leen, para la cantidad de intentos.

Real: números con decimales. Se usó en el ejercicio nueve, porque un promedio casi nunca da exacto.

Carácter: texto. Se usó para la contraseña del ejercicio siete y para la línea de asteriscos del ejercicio cinco.

Lógico: solo puede valer verdadero o falso. Es el tipo de las banderas, y se usó en los ejercicios siete, ocho y diez.

En Python no hace falta declarar el tipo: el lenguaje lo deduce solo. Pero sí tienes que convertir lo que escribe el usuario, y esa es la razón de que veas las palabras `int` y `float` en el código.

Esto es importante: todo lo que el usuario teclea llega al programa como `TEXTO`, aunque haya escrito un número. Si no lo conviertes, el programa intentaría sumar textos en vez de números. La palabra `int` convierte a número entero, y la palabra `float` convierte a número con decimales.

3.8 Leer y Escribir: la entrada y la salida

Leer, en PSeInt, es pedirle un dato al usuario y guardarlo en una variable. En Python se llama `input`.

Escribir, en PSeInt, es mostrar algo en pantalla. En Python se llama `print`.

En Python vas a ver que muchos mensajes llevan una letra `e` pegada antes de las comillas. Eso se llama `e` string, y sirve para meter el valor de una variable dentro de un texto. En lugar de andar pegando pedazos con signos de más, escribes el texto normal y metes la variable entre llaves. Es más corto y se lee mejor.

3.9 Palabras que solo existen en Python

Hay cinco palabras del código en Python que no tienen equivalente directo en el pseudocódigo, y conviene que sepas qué hacen.

`Break`: rompe el ciclo y sale inmediatamente, sin importar la condición. Se usó en el ejercicio cuatro para imitar el Repetir Hasta Que.

`Continue`: se salta el resto de la vuelta actual y va directo a la siguiente. Se usó en el ejercicio diez, para que un intento fuera de rango no se cuente.

`Def`: sirve para crear una función, o sea, un pedacito de programa con nombre propio que puedes usar muchas veces. En estos ejercicios se usó para crear las funciones que leen datos con validación, y así no repetir el mismo código diez veces.

Return: dentro de una función, entrega el resultado y termina la función.

Try y except: sirven para atrapar errores sin que el programa se caiga. Se usaron para que, si el usuario escribe letras donde iba un número, el programa le avise amablemente en lugar de reventar.

Y por último, esa línea al final de cada programa que dice si el nombre es igual a main, entonces llama a main. Su significado práctico es sencillo: ejecuta el programa solo cuando lo corres directamente. Es una convención estándar de Python; puedes copiarla tal cual.

Ejercicio 1: Tabla de multiplicar

Empezamos con el más sencillo, pero no lo subestimes: aquí se establece el molde de todos los demás.

El problema dice: pide un número al usuario y muestra su tabla de multiplicar, del uno al diez.

Aplicando las tres preguntas

Primera pregunta, ¿cuántas veces se repite? Diez veces exactas. Siempre diez, no importa qué número escriba el usuario. Entonces usamos el ciclo Para.

Segunda pregunta, ¿qué necesito recordar entre vueltas? Nada. Cada línea de la tabla es independiente de las demás. Así que no hay acumulador ni contador que inventar.

Tercera pregunta, ¿cómo termina? Cuando el contador pasa de diez. De eso se encarga el propio ciclo Para.

Cómo se construyó

```
Leer numero                <- AFUERA del ciclo
Para i <- 1 Hasta 10 Con Paso 1 Hacer
  resultado <- numero * i
  Escribir numero, " x ", i, " = ", resultado
FinPara
```

La idea central está en la línea del cálculo, y es más profunda de lo que parece: hay dos variables multiplicándose, pero solo una de ellas cambia.

El número que escribió el usuario se queda quieto todo el tiempo. La variable *i*, en cambio, va tomando uno, dos, tres, y así hasta diez. Al multiplicar una fija por una que cambia, sale la tabla completa.

Sigamos la traza con el número siete. En la primera vuelta, *i* vale uno, y siete por uno da siete. En la segunda, *i* vale dos, y siete por dos da catorce. En la tercera, *i* vale

tres, y da veintiuno. Y así hasta la décima vuelta, donde i vale diez y el resultado es setenta.

El error típico

El error clásico es poner la instrucción Leer dentro del ciclo. Si haces eso, el programa te va a pedir el número diez veces seguidas, una por cada vuelta. Y no es que se rompa: simplemente hace algo absurdo.

De ahí sale una regla que te va a servir siempre: lo que se pregunta una sola vez va antes del ciclo. Lo que se repite va adentro. Cada vez que dudes dónde poner una línea, hazte esa pregunta.

Ejercicio 2: Suma de los primeros N números

Si tuviera que elegir un solo ejercicio de los diez para que te lo aprendieras bien, sería este. Porque aquí aparece el acumulador, y el acumulador está en todos los programas del mundo.

El problema dice: pide un número entero positivo, al que llamamos ene, y suma todos los números desde el uno hasta ene.

Aplicando las tres preguntas

Primera pregunta: se repite ene veces, y ene ya lo sé porque el usuario me lo dijo. Entonces, ciclo Para.

Segunda pregunta, y aquí está la novedad: sí necesito recordar algo entre vueltas. Necesito recordar la suma que llevo acumulada. Esa variable es el acumulador.

Cómo se construyó

```
suma <- 0           <- el acumulador nace en cero, AFUERA
Para i <- 1 Hasta n Hacer
  suma <- suma + i   <- el corazón del programa
FinPara
```

Detente en esa línea del centro: suma, flecha, suma más i. Como ecuación de matemáticas sería falsa, porque ningún número es igual a sí mismo más otro. Pero no es una ecuación. Es una orden, y se lee de derecha a izquierda.

Dice así: toma el valor que ya tenía la suma, súmale i, y guarda el resultado otra vez en la suma. Esa frase, aprendértela de memoria, te va a resolver la mitad de los problemas de programación que veas.

Hagamos la traza con ene igual a cinco. Antes de empezar, la suma vale cero. Primera vuelta: i vale uno, cero más uno es uno, y la suma queda en uno. Segunda vuelta: i vale dos, uno más dos es tres, y la suma queda en tres. Tercera vuelta: i vale tres, tres más tres es seis. Cuarta vuelta: i vale cuatro, seis más cuatro es diez. Quinta vuelta: i vale cinco, diez más cinco es quince. Resultado final, quince.

El error típico

Poner la línea suma igual a cero DENTRO del ciclo. Si haces eso, en cada vuelta borras todo lo acumulado, y el resultado final va a ser siempre igual a ene. Con ene igual a cinco te daría cinco en vez de quince.

Vuelvo a la regla del capítulo dos, que es la misma: el acumulador se inicializa una sola vez, antes de entrar al ciclo.

Lo otro que se hizo aquí: validar la entrada

El enunciado pide un número POSITIVO. ¿Y qué pasa si el usuario escribe un cero o un número negativo? Para eso se agregó un ciclo de validación que vuelve a preguntar hasta que el dato sirva.

Y fíjate en el detalle: para validar se usó Repetir Hasta Que, no Mientras. La razón es la lógica de la tercera respuesta de la primera pregunta: hay que preguntar al menos una vez antes de poder juzgar si el dato está bien. Antes de leerlo, no hay nada que juzgar.

Ejercicio 3: Contador regresivo

El problema dice: pide un número y muestra una cuenta regresiva desde ese número hasta cero.

La novedad: el ciclo también puede ir hacia atrás

Hasta ahora todos los ciclos avanzaban sumando. Aquí aprendemos que el contador también puede retroceder.

En PSeInt se logra con las palabras Con Paso menos uno, que significan: en vez de sumar uno en cada vuelta, resta uno.

```
PSeInt: Para i <- n Hasta 0 Con Paso -1 Hacer  
Python: for i in range(n, -1, -1):
```

Los tres números del range de Python

En Python, la palabra range lleva tres números, y conviene entenderlos bien porque confunden a todo el mundo al principio.

El primero dice desde dónde empieza. El segundo dice hasta dónde llega, pero SIN incluirlo. Y el tercero dice de cuánto en cuánto avanza.

Ese segundo número es la trampa. Range nunca incluye su valor final. Se detiene justo antes.

Y aquí está el detalle fino de este ejercicio: el enunciado pide llegar hasta el cero, incluyéndolo. Como range no incluye su final, hay que escribir menos uno para que el cero sí alcance a salir.

El error típico

Escribir range de ene, cero, menos uno. Con eso el programa cuenta cinco, cuatro, tres, dos, uno, y se detiene ahí. Nunca imprime el cero, que era justo lo que pedía el enunciado. Es un error que no marca ningún fallo: el programa corre perfecto, solo que le falta el último número. Esos son los errores más difíciles de encontrar.

Ejercicio 4: Acumulado hasta cero

El problema dice: pide números al usuario y ve sumándolos. El ciclo termina únicamente cuando el usuario escriba un cero. Al final, muestra el total.

Aplicando las tres preguntas

Primera pregunta, y aquí cambia todo: ¿cuántas veces se repite? No lo sé. Pueden ser dos números o doscientos, depende completamente del usuario. Pero sí sé una cosa: tengo que leer al menos un número antes de poder decidir si paro.

Esa combinación —no sé cuántas veces, pero seguro es al menos una— es exactamente la que pide el ciclo Repetir Hasta Que.

El concepto nuevo: el centinela

El cero, en este programa, se llama centinela. Un centinela es un valor que no forma parte de los datos reales, y que solo sirve como señal de que el usuario ya terminó.

```
total <- 0
Repetir
  Leer numero
  Si numero <> 0 Entonces
    total <- total + numero
  FinSi
Hasta Que numero = 0
```

Fíjate en esa condición de adentro, la que dice si el número es distinto de cero. Está ahí porque el centinela NO se debe sumar. Solo detiene el ciclo, no forma parte del total.

Ahora, tal vez estés pensando: pero sumar cero no cambiaría nada, el total quedaría igual. Y tienes razón en cuanto al total. Pero el programa también cuenta cuántos números se ingresaron, y ahí sí importaría: contaría un número de más, uno que el usuario nunca quiso incluir.

Cómo se imita el Repetir Hasta Que en Python

Python no tiene el ciclo Repetir Hasta Que. Se imita con una estructura de dos partes que vale la pena memorizar.

Se escribe `while true`, que significa literalmente repite para siempre. Y adentro, en el punto donde debería estar la condición de salida, se pone un `if` con la palabra `break`, que rompe el ciclo y sale.

```
while True:                # repite para siempre...
    numero = leer_entero("Numero: ")
    if numero == 0:
        break              # ...hasta que se cumpla la salida
    total += numero
```

El resultado es idéntico: el cuerpo se ejecuta al menos una vez, y la condición se evalúa al final. Es la traducción exacta.

El error típico

Usar el ciclo Mientras y poner la condición antes de leer. En la primera vuelta, la variable `número` todavía no tiene ningún valor guardado, así que el programa falla antes de empezar. Es un recordatorio de por qué la primera pregunta importa tanto: elegir mal el ciclo no es cuestión de estilo, rompe el programa.

Ejercicio 5: Impresión de patrones

Este es el más difícil de los cinco básicos, porque aquí aparecen los ciclos anidados, o sea, un ciclo metido dentro de otro.

El problema dice: pide un número *n*, e imprime *n* líneas. La primera con un asterisco, la segunda con dos, la tercera con tres, y así hasta llegar a *n* asteriscos.

Por qué hacen falta dos ciclos

Piénsalo así: necesitas repetir dos cosas distintas al mismo tiempo. Por un lado, repetir líneas. Por otro, dentro de cada línea, repetir asteriscos. Dos repeticiones distintas piden dos ciclos, uno metido dentro del otro.

```
Para i <- 1 Hasta n Hacer      <- EXTERNO: cuenta las filas
  linea <- ""                  <- se vacia en cada fila
  Para j <- 1 Hasta i Hacer    <- INTERNO: cuenta las columnas
    linea <- Concatenar(linea, "*")
  FinPara
  Escribir linea              <- se imprime al terminar la fila
FinPara
```

La regla de oro de los ciclos anidados

Memoriza esta frase: el ciclo interno se ejecuta COMPLETO en cada vuelta del externo. No una vuelta del interno por cada vuelta del externo; el interno entero, de principio a fin, cada vez.

El ciclo externo controla las filas. El interno controla las columnas. Esa división vale para todo lo que hagas con tablas y matrices en el futuro.

El truco que hace crecer el triángulo

Mira el límite del ciclo interno. No dice hasta un número fijo: dice hasta *i*, que es la variable del ciclo externo. Ese detalle es todo el ejercicio.

En la primera vuelta del externo, *i* vale uno, así que el interno da una sola vuelta y sale un asterisco. En la segunda, *i* vale dos, el interno da dos vueltas y salen dos

asteriscos. En la tercera, tres asteriscos. Cada fila es más larga que la anterior porque su límite creció.

El error típico, y es muy visual

La línea que vacía la variable está dentro del ciclo externo, pero fuera del interno. Esa posición es exacta, no es casualidad: significa vaciar una vez por cada fila.

Si por error la pones antes del ciclo externo, la línea nunca se vacía, y los asteriscos se van acumulando. En vez de un triángulo saldría uno, luego tres, luego seis, luego diez asteriscos. El resultado se ve raro de inmediato, y por eso este error es fácil de detectar. Aprovechalo: es una buena forma de entender qué significa realmente la posición de una línea.

Un dato curioso para cerrar. ¿Cuántos asteriscos se imprimen en total? Uno más dos más tres, hasta ene. O sea, exactamente la suma del ejercicio dos. Los ejercicios se conectan entre sí más de lo que parece.

Ejercicio 6: Clasificador de pares e impares

Entramos al nivel intermedio. A partir de aquí, todos los ejercicios combinan ciclos con decisiones.

El problema dice: pregunta cuántos números va a ingresar el usuario, léelos uno por uno, y cuenta cuántos son pares, cuántos impares y cuántos son iguales a cero.

Las tres preguntas

Primera: sé cuántas veces, porque el usuario me lo dijo al principio. Ciclo Para.

Segunda: necesito recordar tres cosas, y las tres son contadores. Cuántos pares llevo, cuántos impares y cuántos ceros. Los tres nacen en cero, afuera del ciclo.

La herramienta nueva: el operador MOD

Ya lo vimos en el diccionario, pero repasémoslo porque aquí es el protagonista. MOD devuelve el residuo de una división. Si un número entre dos no deja residuo, es par. Si deja residuo uno, es impar.

```
Si numero = 0 Entonces      <- el cero se pregunta PRIMERO
  ceros <- ceros + 1
SiNo
  Si numero MOD 2 = 0 Entonces
    pares <- pares + 1
  SiNo
    impares <- impares + 1
  FinSi
FinSi
```

Lo más importante de este ejercicio: el orden de las preguntas

Aquí hay una sutileza que decide si el programa está bien o mal. El cero es un número par: cero entre dos da cero de residuo. Entonces, si preguntáramos primero si es par, el cero caería en el grupo de los pares y jamás llegaría a su propio contador.

Por eso el cero se aparta ANTES que todo lo demás. La regla general que sacas de aquí es: cuando tengas varios casos y uno sea especial, pregunta primero por el especial.

Por qué SiNo y no tres Si sueltos

Fíjate en que las decisiones están encadenadas con SiNo, que en Python se escribe elif. Eso significa que en cuanto una condición se cumple, las demás ni siquiera se evalúan. Cada número entra exactamente en un contador, ni más ni menos.

Si en cambio escribieras tres Si independientes, el cero se contaría dos veces: una en ceros y otra en pares. Y notarías el error porque la suma de los tres contadores no coincidiría con el total de números ingresados. Ese, por cierto, es un buen truco para verificar tus propios programas: comprueba que las partes sumen el total.

Ejercicio 7: Validación de contraseña

El problema dice: pide una contraseña, permite un máximo de tres intentos. Si acierta antes de agotarlos, muestra acceso concedido y termina. Si no, muestra acceso bloqueado.

El concepto nuevo: la bandera

Una bandera es una variable que solo puede valer verdadero o falso, y que responde a una pregunta de sí o no. Aquí la pregunta es: ¿ya acertó el usuario?

La bandera nace valiendo falso, porque al empezar nadie ha acertado nada. Y cambia a verdadero en el momento exacto en que el usuario escribe bien la contraseña. Se dice que la bandera se levanta.

```
intentos <- 0
acceso <- Falso          <- la bandera

Mientras intentos < maxIntentos Y acceso = Falso Hacer
  Leer intento
  intentos <- intentos + 1      <- nunca olvidar
  Si intento = claveCorrecta Entonces
    acceso <- Verdadero        <- se levanta
  FinSi
FinMientras
```

Por qué el conector es Y, y no O

Esta es la parte que hay que entender de verdad, y es la razón por la que el capítulo tres dedicó tanto espacio a los conectores.

El ciclo debe continuar mientras se cumplan dos cosas a la vez: que queden intentos disponibles, Y que el usuario todavía no haya acertado. Con el conector Y, basta con que una de las dos falle para salir. Si acierta, salimos. Si se agotan los intentos, también salimos. Las dos salidas funcionan.

Ahora imagina que usas el conector O por error. En el momento en que el usuario acierta, la segunda condición se vuelve falsa, pero la primera sigue siendo verdadera porque aún quedan intentos. Y al conector O le basta con que una sea verdadera.

Resultado: el ciclo continuaría, y el programa te seguiría pidiendo la contraseña después de haberte dejado entrar.

Lo grave de este error es que el programa no marca ningún fallo. Corre perfectamente, solo que hace algo absurdo. Por eso elegir bien el conector no es un detalle.

Para qué sirve la bandera al final

Cuando el ciclo termina, hay un problema: no sabemos por cuál de las dos razones salió. ¿Salió porque acertó, o porque se le acabaron los intentos? El programa necesita saberlo para mostrar el mensaje correcto.

Y ahí está el verdadero propósito de la bandera: guarda esa información. Después del ciclo, basta con preguntarle. Si la bandera está levantada, acertó. Si sigue en falso, se agotaron los intentos.

El error típico

Olvidar la línea que aumenta los intentos. Si no la pones, la condición nunca cambia y el programa te pide la contraseña para siempre. Vuelve a la tercera pregunta del método: siempre busca, dentro del ciclo, la línea que modifica la condición de salida. Si no existe, tienes un ciclo infinito.

Ejercicio 8: Números primos en un rango

Este es el ejercicio más completo de los diez. Junta ciclos anidados, banderas y una idea matemática bonita.

El problema dice: pide dos números que definen un rango, recorre ese rango, y muestra únicamente los que sean primos.

Punto de partida: qué es un número primo

Un número es primo si solo se puede dividir exactamente entre uno y entre sí mismo. El siete es primo. El nueve no lo es, porque también se divide entre tres.

Y de esa definición sale la estructura del programa. Para saber si un número es primo hay que probar divisores, y probar divisores requiere un ciclo. Pero además hay que hacer eso con cada número del rango, y eso requiere otro ciclo por fuera. Dos ciclos anidados.

La estrategia: presunción de inocencia

El programa asume que el número SÍ es primo, y luego busca un contraejemplo. Si encuentra un solo divisor, la bandera cae a falso y ya está: el número queda descartado.

```
esPrimo <- Verdadero      <- se ASUME primo...
divisor <- 2
Mientras divisor * divisor <= numero Y esPrimo = Verdadero Hacer
  Si numero MOD divisor = 0 Entonces
    esPrimo <- Falso      <- ...hasta demostrar lo contrario
  FinSi
  divisor <- divisor + 1
FinMientras
```

Fíjate en que la bandera también aparece en la condición del ciclo. Eso hace que en cuanto se encuentre un divisor, el ciclo se detenga de inmediato en vez de seguir probando en vano. Es una optimización pequeña pero elegante.

La parte bonita: por qué basta llegar a la raíz cuadrada

Aquí está la idea que hace valioso a este ejercicio. Los divisores de un número siempre vienen en parejas. Vamos a verlo con el treinta y seis.

Uno por treinta y seis. Dos por dieciocho. Tres por doce. Cuatro por nueve. Seis por seis. Y después de ahí... nueve por cuatro, doce por tres, dieciocho por dos. Las mismas parejas, pero al revés.

El punto donde se voltean es el seis, que es exactamente la raíz cuadrada de treinta y seis. Y esto pasa siempre, con cualquier número.

La conclusión es poderosa: si un número tuviera un divisor mayor que su raíz cuadrada, su pareja sería menor que la raíz, y ya la habríamos encontrado antes. Entonces no hace falta buscar más allá de la raíz.

Para el número noventa y siete, eso significa probar nueve divisores en lugar de noventa y cinco. Con números grandes la diferencia es enorme, y es la razón por la que este programa no se tarda.

Por qué se multiplica en vez de sacar la raíz

El código no calcula la raíz cuadrada: escribe divisor por divisor menor o igual que el número. Es la misma condición, dicha al revés, y tiene dos ventajas. Multiplicar es rápido y exacto, mientras que la raíz cuadrada da decimales y obliga a redondear, y los redondeos introducen errores sutiles. Es un buen ejemplo de cómo un programador reescribe una idea matemática para que sea más segura.

El error típico

Empezar el divisor en uno en lugar de en dos. Como absolutamente todo número es divisible entre uno, la bandera caería siempre y ningún número saldría primo. El programa correría sin marcar error y no imprimiría nada.

Y un detalle más que se cuidó: los números menores que dos, o sea el cero, el uno y los negativos, no son primos por definición. Por eso se apartan antes de entrar al ciclo, igual que se apartó el cero en el ejercicio seis.

Ejercicio 9: Promedio con filtrado

Este ejercicio junta el acumulador del ejercicio dos con el centinela del ejercicio cuatro, y le agrega un filtro.

El problema dice: el usuario ingresa calificaciones una por una, y se detiene al escribir un número negativo. Hay que calcular el promedio, pero ignorando las notas menores a cinco, que son las reprobatorias.

El patrón de la doble lectura

Aquí aparece algo que confunde a mucha gente: el programa tiene dos instrucciones Leer, no una. Y no es un error, es necesario.

```
Leer nota          <- PRIMERA lectura, antes del ciclo
Mientras nota >= 0 Hacer
  ... procesar la nota ...
  Leer nota        <- SIGUIENTE lectura, al final
FinMientras
```

La razón es la naturaleza del ciclo Mientras: evalúa la condición ANTES de entrar. Entonces la variable ya tiene que traer un valor la primera vez, o no habría nada que evaluar. Por eso hay una lectura antes del ciclo, para arrancar, y otra al final del cuerpo, para preparar la siguiente comprobación.

Ese es el precio de usar Mientras en vez de Repetir Hasta Que. Compáralo con el ejercicio cuatro, donde con Repetir Hasta Que bastaba una sola lectura. Ninguna de las dos formas es mejor: cada ciclo tiene su costo, y saber cuál es te ayuda a elegir.

Lo que califica este ejercicio: suma y contador van juntos

Un promedio es una suma dividida entre una cantidad. Entonces, si una nota entra en la suma, tiene que entrar también en la cuenta. Y si se ignora, debe quedar fuera de las dos.

```
Si nota >= 5 Entonces
  suma <- suma + nota    <- estas dos lineas
  contadas <- contadas + 1 <- SIEMPRE van juntas
FinSi
```

Veámoslo con números. Supón que el usuario ingresa ocho, tres, diez y seis. La nota de tres es reprobatoria y se ignora. Entonces la suma es ocho más diez más seis, que da veinticuatro, y las notas contadas son tres. Veinticuatro entre tres da ocho punto cero cero. Ese es el resultado correcto.

Ahora, si por descuido contaras también la nota reprobatoria, dividirías veinticuatro entre cuatro y te daría seis. Un promedio equivocado, y equivocado hacia abajo, que además perjudicaría al estudiante.

La protección contra la división entre cero

Antes de dividir, el programa verifica que el contador sea mayor que cero.

¿Por qué? Porque puede pasar que todas las notas hayan sido reprobatorias. En ese caso el contador quedaría en cero, y dividir entre cero no da un número raro: hace que el programa se caiga con un error.

Grábate esta regla, porque aplica a cualquier programa que escribas en tu vida: antes de dividir, verifica que el divisor no sea cero. Es de las comprobaciones más baratas y de las que más problemas evitan.

Ejercicio 10: Juego de adivinanza

Cerramos con el más divertido, y también el que esconde la idea más profunda de todo el conjunto.

El problema dice: genera un número secreto entre uno y cien, y pide intentos hasta que el usuario adivine. En cada intento fallido, dile si el número buscado es mayor o menor que el que escribió.

Un ciclo verdaderamente indefinido

Este es el único ejercicio donde no hay ningún límite de vueltas. Puede terminar en un intento con muchísima suerte, o en cuarenta con muy poca. El ciclo simplemente sigue mientras la bandera diga que todavía no acierta.

La condicional de tres ramas

Al comparar dos números solo hay tres posibilidades: son iguales, el primero es menor, o el primero es mayor. El programa da una respuesta distinta para cada una.

```
Si intento = secreto Entonces
  acerto <- Verdadero
SiNo
  Si intento < secreto Entonces
    Escribir "el numero buscado es MAYOR que ", intento
  SiNo
    Escribir "el numero buscado es MENOR que ", intento
  FinSi
FinSi
```

Cuidado con la dirección de la pista

Aquí está el error más común de este ejercicio, y es puramente de razonamiento. Si tu intento es MENOR que el secreto, el mensaje correcto es que el número buscado es MAYOR.

Se siente al revés, y por eso mucha gente lo invierte. La clave es recordar que la pista habla del número secreto, no del tuyo. Tú dijiste veinticinco y el secreto es

cuarenta y dos: entonces el secreto es mayor que lo que dijiste. Léelo siempre desde el punto de vista del número escondido.

El intento que no cuenta

El programa valida que el intento esté entre uno y cien. Y si está fuera, lo rechaza SIN gastar un intento al usuario. Eso se logra poniendo la línea que aumenta los intentos después de la validación, no antes.

Es una decisión de diseño, no un accidente. Y muestra algo importante: la posición de una línea dentro de un programa comunica una intención. Aquí la intención es ser justo con el usuario.

La idea profunda: la búsqueda binaria

Terminemos con lo mejor. ¿Cuál es la mejor estrategia para ganar este juego?

La respuesta es partir el rango a la mitad cada vez. Empiezas diciendo cincuenta. Si te dicen que es menor, ya eliminaste la mitad de las opciones de un solo golpe: ahora sabes que está entre uno y cuarenta y nueve. Dices veinticinco. Si te dicen que es mayor, queda entre veintiséis y cuarenta y nueve. Y así.

Cuenta cuántos pasos hacen falta. De cien posibilidades pasas a cincuenta, luego veinticinco, luego trece, luego siete, luego cuatro, luego dos, luego una. Siete pasos. Con esa estrategia, siempre adivinas en siete intentos como máximo, sin importar cuál sea el número.

Eso se llama búsqueda binaria, y no es un truco de juego: es uno de los algoritmos más importantes que existen. Es el método que usan las bases de datos para encontrar un registro entre millones en cuestión de milisegundos.

Piénsalo un momento. Con veinte preguntas de este tipo podrías encontrar un dato entre un millón. Con treinta, entre mil millones. Y ese poder sale de una idea que cabe en una frase: cada pregunta elimina la mitad.

Así que este jueguito de adivinanzas y el buscador de una base de datos profesional son, por dentro, exactamente el mismo algoritmo. Esa es la clase de conexión que hace que valga la pena estudiar programación.

Repaso final: los diez errores que debes evitar

Llegamos al final. Para cerrar, te dejo el repaso más útil que existe: la lista de los errores típicos, uno por ejercicio. Si evitas estos diez, evitas la mayoría de los problemas.

Uno. En la tabla de multiplicar: poner la lectura del dato dentro del ciclo. Lo que se pregunta una sola vez va afuera.

Dos. En la suma de uno hasta ene: inicializar el acumulador dentro del ciclo. Se inicializa una sola vez, antes de entrar.

Tres. En el contador regresivo: olvidar que el range de Python no incluye su valor final, y quedarse sin imprimir el cero.

Cuatro. En el acumulado hasta cero: usar el ciclo Mientras cuando hacía falta leer al menos una vez antes de decidir.

Cinco. En el patrón de asteriscos: no vaciar la línea en cada fila, y terminar con los asteriscos acumulados.

Seis. En el clasificador: no apartar el cero antes de preguntar si es par, y contarlo dos veces.

Siete. En la contraseña: olvidar aumentar el contador de intentos, y crear un ciclo infinito. O confundir el conector Y con el conector O.

Ocho. En los números primos: empezar a probar divisores desde el uno, con lo cual ningún número resulta primo.

Nueve. En el promedio: contar las notas que se supone que hay que ignorar, o dividir sin verificar que el contador no sea cero.

Diez. En el juego de adivinanza: invertir la dirección de la pista, y decir mayor cuando había que decir menor.

Y si solo te llevas tres ideas

Primera: antes de escribir, hazte las tres preguntas. ¿Cuántas veces se repite? ¿Qué necesito recordar entre vueltas? ¿Cómo termina?

Segunda: las variables que recuerdan algo —acumuladores, contadores y banderas— se crean y se ponen en cero antes del ciclo, nunca adentro.

Tercera: cuando uses conectores, el Y exige que se cumplan todas las condiciones, y el O se conforma con una sola. Elegir mal entre ellos produce programas que corren sin marcar error pero hacen algo absurdo, y esos son los más difíciles de encontrar.

Con eso terminamos. Ahora escucha esto una segunda vez mientras vas siguiendo el código escrito, y vas a ver que todo encaja. Mucha suerte con tu trabajo.